Lecture 5

Quantum computers are supposed to be very powerful, but can they actually compute simple Boolean functions like AND?

Quantum AND?

Is there a 2-qubit unitary that computes the logical AND? Given $x, y \in \{0,1\}$, it should output $\text{AND}(x,y)$ on the first qubit:



What should be the output on the second qubit? Let's try some arbitrary states $|\psi_i\rangle$:

$$|00\rangle \mapsto |0\rangle|\psi_1\rangle$$
$$|01\rangle \mapsto |0\rangle|\psi_2\rangle$$
$$|10\rangle \mapsto |0\rangle|\psi_3\rangle$$
$$|11\rangle \mapsto |1\rangle|\psi_4\rangle$$

But a unitary gate must preserve orthogonality, so the three output states $|\psi_1\rangle, |\psi_2\rangle, |\psi_3\rangle \in \mathbb{C}^2$ when $|0\rangle$ is in the first register must be orthogonal. But this is impossible since we only have two dimensions.

Are we doomed? Are quantum computers not even able to do what classical computers can? This would be very unfortunate and make them hard to compare. Luckily, this is not the case, but we need to explicitly address this issue. We will then come back to the quantum AND example and see how to resolve it.

Reversibility

The main problem we encounter when trying to run a classical algorithm on a quantum computer is reversibility.

Recall that a quantum algorithm is a sequence of unitary operations and that any unitary operation is invertible: $U^{-1} = U^\dagger$. This means that we can undo any operation and go back to the previous state. E.g., if the state at step 3 is $U_3 U_2 U_1 |0\rangle$, we can apply $U_3^\dagger$ to get back to

$$U_3^\dagger U_3 U_2 U_1 |0\rangle = U_2 U_1 |0\rangle,$$

which is the state after step 2. This property is called reversibility. As a consequence of it, for example, we cannot erase information because that would prohibit us from going back. This makes it seem like quantum computation is more restrictive than deterministic or probabilistic computation which include non-reversible operations. Luckily, this is not a problem since it turns out that we can implement all non-reversible operations in a reversible way. Let us figure out how this works for deterministic operations. In other words, let's figure out how to compute any Boolean function in a reversible way.

Computing a Boolean function reversibly

Let $f:\{0,1\}^n \to \{0,1\}$ be an n-argument Boolean function. Clearly, the map

$$|x\rangle \longmapsto |f(x)\rangle$$

in general is not reversible and hence not unitary. A simple way of making any deterministic map reversible is by keeping the input:

$$|x\rangle \longmapsto |x, f(x)\rangle.$$

While this is reversible, it is not unitary since the output dimension is larger than the input dimension. We can make the dimensions match by adding a dummy $|0\rangle$ to the input:

$$|x, 0\rangle \longmapsto |x, f(x)\rangle$$

Is this unitary? More specifically, can this be implemented by a permutation matrix, i.e., an invertible linear operation that maps standard basis states to standard basis states (such deterministic operation is a special case of a unitary operation). The above map is still not good enough for this because

- it is not fully defined (what is the image of $|x,1\rangle$?),
- nor is its inverse (what is the pre-image of $|x,y\rangle$ when $y \neq f(x)$?).

An elegant way to get around both issues and turn f into a permutation (and hence also a unitary) is by defining

$$U_f\,|x,y\rangle = |x, y \oplus f(x)\rangle$$

for all $x \in \{0,1\}^n$ and $y \in \{0,1\}$. Note that $U_f^2 = I$, i.e., U_f is self-inverse since $U_f^2\,|x,y\rangle = |x, y \oplus f(x) \oplus f(x)\rangle = |x,y\rangle$. We will call U_f the bit oracle for function f.

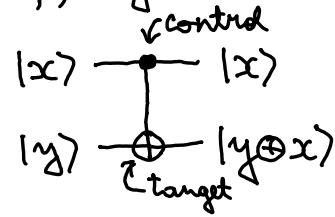
Controlled gates

Now we know that any $f:\{0,1\}^n \to \{0,1\}$ can in principle be made reversible. But how do we actually implement it as a sequence of elementary reversible steps? That is, given a logical circuit for f in terms of elementary logical operations, how do we implement each step reversibly? Here are some useful gates for this.

CNOT or <u>controlled-NOT</u> is a 2-qubit gate that flips the second bit (target) if the first bit (control) is in state $|1\rangle$: $\mathrm{CNOT}|x,y\rangle = |x, y\oplus x\rangle$, $x,y\in\{0,1\}$.

$$|00\rangle \mapsto |00\rangle$$
$$|01\rangle \mapsto |01\rangle$$
$$|10\rangle \mapsto |11\rangle$$
$$|11\rangle \mapsto |10\rangle$$

$$\mathrm{CNOT} = \begin{pmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&0&0&1\\ 0&0&1&0 \end{pmatrix}$$



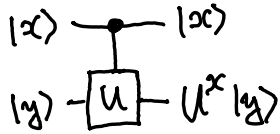
Note that $\mathrm{CNOT} = \underbrace{U_f}_{\text{(bit oracle)}}$ for the identity function $f(x)=x$, $x\in\{0,1\}$. Also, $\mathrm{CNOT}|x,0\rangle = |x,x\rangle$, so it <u>copies in the standard basis</u>. However, CNOT does not copy an arbitrary state, i.e., generally, $\mathrm{CNOT}|\psi\rangle\otimes|0\rangle \neq |\psi\rangle\otimes|\psi\rangle$.

It is useful to notice that $\mathrm{CNOT} = |0\rangle\langle 0|\otimes I + |1\rangle\langle 1|\otimes X$, where $X = \mathrm{NOT} = \left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$. Intuitively, this equation says: if qubit 1 is $|0\rangle$, do nothing (or apply I) to qubit 2, and if qubit 1 is $|1\rangle$ then invert qubit 2 (apply X).

More generally, if U is any single-qubit unitary then the <u>controlled-U</u> gate maps $|x\rangle\otimes|y\rangle \mapsto |x\rangle\otimes U^x|y\rangle$, $x,y\in\{0,1\}$.

$$|00\rangle \mapsto |00\rangle$$
$$|01\rangle \mapsto |01\rangle$$
$$|10\rangle \mapsto |1\rangle\otimes U|0\rangle$$
$$|11\rangle \mapsto |1\rangle\otimes U|1\rangle$$

$$\begin{pmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&0&U_{00}&U_{01}\\ 0&0&U_{10}&U_{11} \end{pmatrix}$$

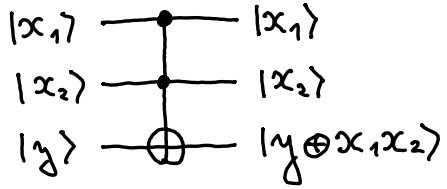


$U^0 = I$
$U^1 = U$

Toffoli gate

This three (qu)bit gate flips the last bit only when the first two bits are both in state 1:

$|x_1, x_2, y\rangle \mapsto |x_1, x_2, y \oplus x_1 x_2\rangle$, for all $x_1, x_2, y \in \{0,1\}$.

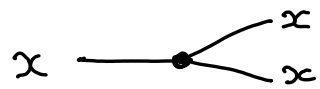


Note that:

- this is the same as controlled–controlled-NOT (NOT on the third qubit, controlled on the first two qubits),
- since $\text{AND}(x_1, x_2) = x_1 x_2$, this is the bit oracle U_{AND} for the AND function (hence also a reversible implementation of the AND gate).

An important fact about the Toffoli gate is that it is universal for reversible classical computation, meaning that by having access to enough 0 and 1 bits and setting the inputs of all Toffoli gates appropriately, you can implement AND, NOT, and FANOUT where

$$\text{FANOUT}(x) = (x, x)$$



By combining these you can implement any logical circuit and hence any Boolean function reversibly!

Say, if you want to implement XOR, notice that

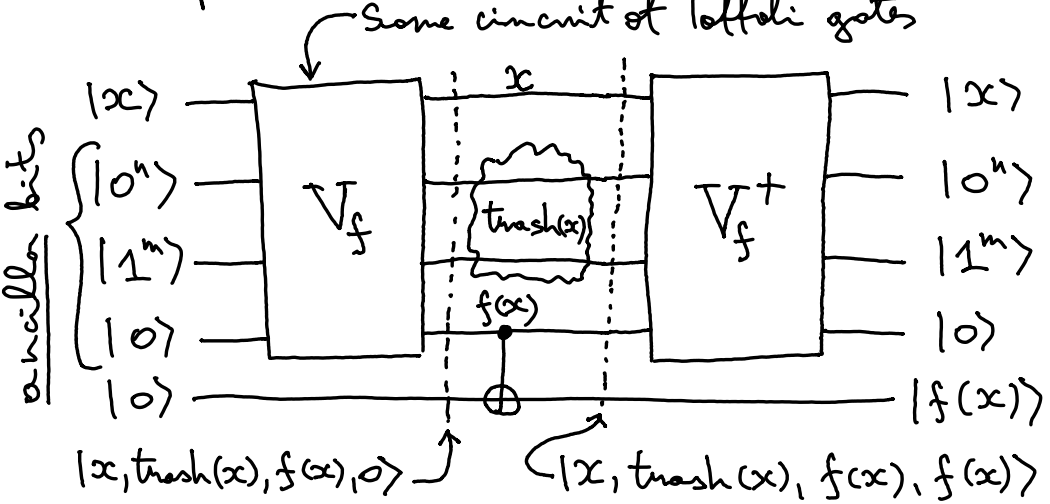


Getting rid of trash...

There is still one more problem: using the Toffoli gate to implement elementary logical gates will generally consume some amount of 0's and 1's and produce various unwanted garbage bits that store intermediate results of the computation to guarantee that it is reversible. In other words, what we end up implementing amounts to something like

$$|x, \underbrace{0^n 1^m}_{\substack{n \text{ copies of } 0 \\ \text{and } m \text{ copies of } 1}}, 0\rangle \xmapsto{V_f} |x, \underbrace{\text{trash}(x)}_{\substack{\text{some trash} \\ \text{that depends on } x}}, f(x)\rangle$$

Can we get rid of trash(x) and return these bits to $0^n 1^m$ or is it fundamentally unavoidable? Surprisingly, we can copy the answer and uncompute the trash:



This way you can reversibly implement the bit oracle U_f for any Boolean function f.

Phase kick-back

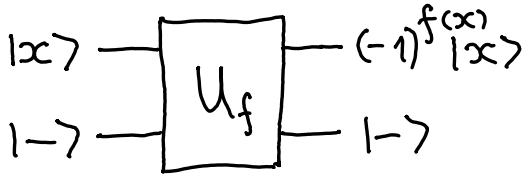
Note that $X|-\rangle = \frac{1}{\sqrt{2}}(X|0\rangle - X|1\rangle) = \frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = -|-\rangle$.
Since $X^0 = I$, we have $X^x|y\rangle = |y \oplus x\rangle$ and we get

$$X^x|-\rangle = (-1)^x|-\rangle$$

for any $x \in \{0,1\}$. More generally, for any bit oracle U_f, $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ for all $x \in \{0,1\}^n$ and $y \in \{0,1\}$ and so

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$$

which is known as phase kick-back. Pictorially,



Since the last qubit is returned back in the same state, we can think of this as using the bit oracle U_f and an extra ancillary qubit $|-\rangle$ to simulate the phase oracle

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle \qquad O_f = \begin{pmatrix} (-1)^{f(00\ldots0)} & & & \\ & (-1)^{f(00\ldots1)} & & \\ & & \ddots & \\ & & & (-1)^{f(11\ldots1)} \end{pmatrix}$$

which is a diagonal n-qubit operation with $(-1)^{f(x)}$ as the x-th entry, $x \in \{0,1\}^n$, on the diagonal.

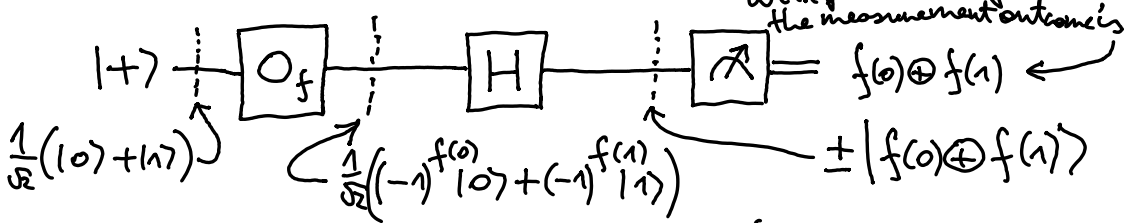
In summary, by setting the last qubit of the bit oracle U_f to be $|-\rangle$, we can implement the phase oracle O_f. Because of this, we will often assume that we are directly given the phase oracle.

Deutsch's algorithm for XOR

Let's say you are given two bits, $x_0, x_1 \in \{0,1\}$ and you want to compute $\mathrm{XOR}(x_0, x_1) = x_0 \oplus x_1$. Let's say each of these bits itself is the result of some other (possibly very difficult) computation:

$x_0 = f(0)$ and $x_1 = f(1)$ for some Boolean function $f: \{0,1\} \to \{0,1\}$. We want to know whether $f(0) \oplus f(1)$ is 0 or 1, i.e., whether $f(0) = f(1)$ or $f(0) \neq f(1)$. The only way to find this out on a classical computer is by computing both $f(0)$ and $f(1)$ first and then XORing the results. Hence, classically two evaluations of f are required. Surprisingly, quantumly only one evaluation of f suffices. Recall that quantum evaluation of f corresponds to applying the oracle U_f or O_f.

Here is Deutsch's algorithm for computing $f(0) \oplus f(1)$ with one application of O_f:



$$= (-1)^{f(0)} \frac{1}{\sqrt{2}} \left(|0\rangle + (-1)^{f(0) \oplus f(1)} |1\rangle \right)$$

$$= (-1)^{f(0)} \begin{cases} |+\rangle & \text{if } f(0) \oplus f(1) = 0 \\ |-\rangle & \text{if } f(0) \oplus f(1) = 1 \end{cases}$$

$$\left(\text{up to global phase}\right) \sim |(-1)^{f(0) \oplus f(1)}\rangle$$

$$H |(-1)^x\rangle = |x\rangle \qquad x \in \{0,1\}$$

Interference

What gives Deutsch's algorithm advantage over (probabilistic) classical algorithms? It is interference – amplitudes with opposite signs can cancel each other out. This can not happen with probabilities since they cannot be negative.

$$H|+\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\cdot\frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}1+1\\1-1\end{pmatrix} = \begin{pmatrix}1\\0\end{pmatrix}$$

$$H|-\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\cdot\frac{1}{\sqrt{2}}\begin{pmatrix}1\\-1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}1-1\\1+1\end{pmatrix} = \begin{pmatrix}0\\1\end{pmatrix}$$

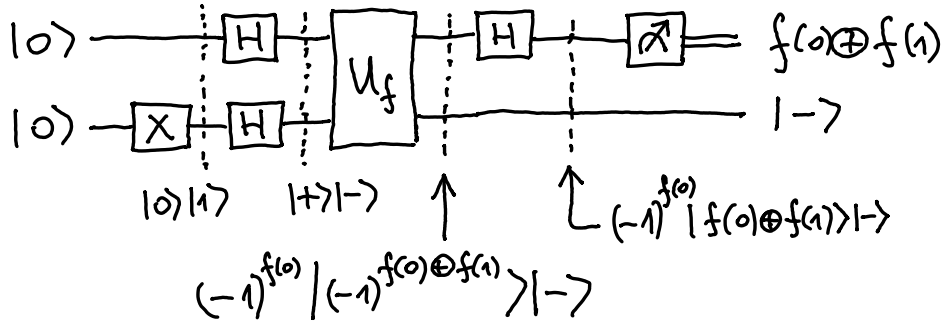
For probabilistic bits we get:

$$\frac{1}{2}\begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}\cdot\frac{1}{2}\begin{pmatrix}1\\1\end{pmatrix} = \frac{1}{4}\begin{pmatrix}1+1\\1+1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}1\\1\end{pmatrix}$$

← no destructive interference

Deutsch's algorithm (full version)

The full version (starting from $|0\rangle|0\rangle$ and using the bit oracle U_f) is as follows:



This algorithm uses only one call to the subroutine U_f for evaluating f and it succeeds in computing the value of $f(0)\oplus f(1)$ with probability 1.

We can return the last qubit back to $|0\rangle$ by applying H and then X.